

CANDY DIALOGUE ENGINE

Basic Dialogue

1.0.0

This guide describes how to create a basic dialogue in the Candy Dialogue Creator, and test it in Godot.

If you haven't done so already, follow the Basic Setup guide.

1. Dialogue Writing

Candy DE Basics

You can think of Candy DE as having three layers: the dialogues, which you write; the engine, which reads dialogue lines; and various UI elements, which display or play dialogue text and media to the player.

The engine processes dialogues one line at a time, and each line acts as an instruction that the engine follows: display text on screen, play audio, modify a variable, display player choices, etc.

Depending on the line, the engine might defer to UI elements. For example, when it processes a spoken line (a character speaking), it does a bunch of internal operations to prepare the line for display, then calls upon the dialogue box to actually display the line on screen; when the engine processes a media command, it calls upon a media player (e.g. `AudioStreamPlayer`) to play the media; and so on.

It's a very simple design:

Dialogues → Engine → UI Elements

Dialogue Basics

Structure

Dialogues are built hierarchically: Conversations contain Blocks, and Blocks contain Lines.

You can write all of your game's dialogues in a single dialogue file, or you can break dialogues into multiple files if you find this more convenient. To illustrate:

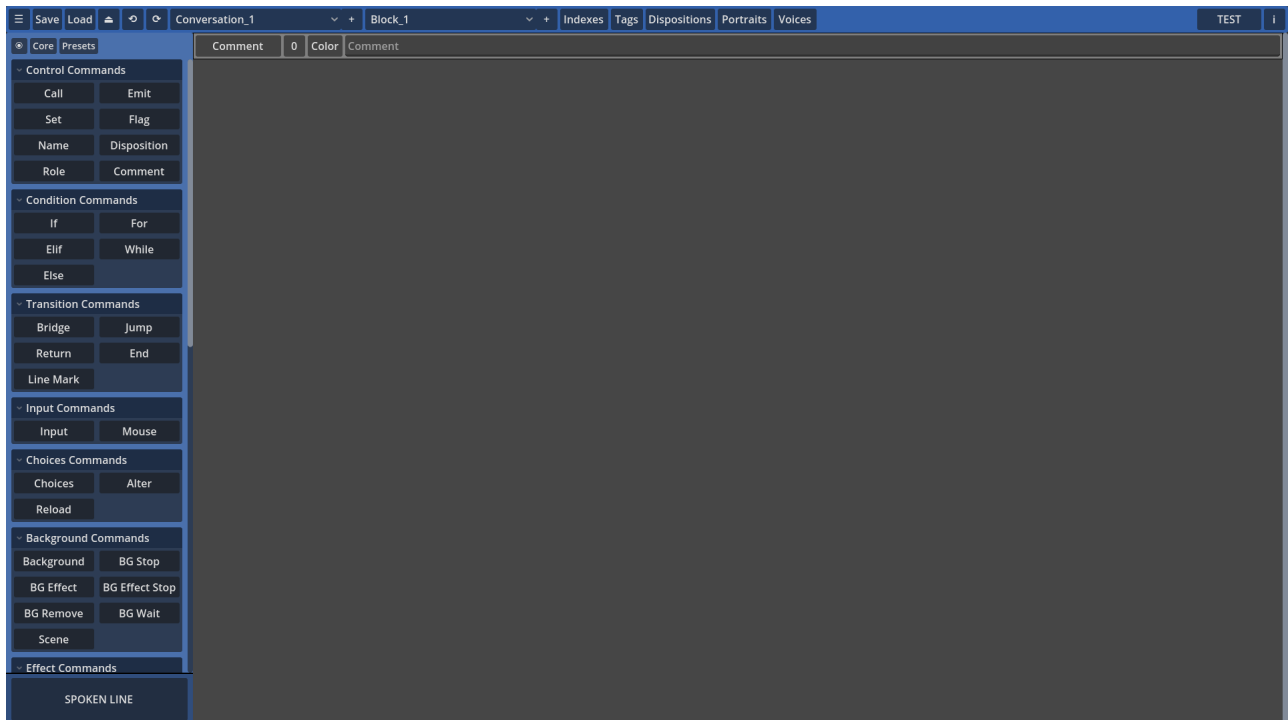
Dialogue Files → Conversations → Blocks → Lines

There are no hard rules about splitting your dialogues into Conversations or Blocks. Generally, you would treat each Conversation as a single scene, while Blocks would be used to strategically separate parts of a scene.

Lines

Two types of lines exist: spoken lines displayed on screen as character speech, and command lines which have various effects on dialogues or on your game, such as modifying variables or calling functions, transitioning to other parts of a dialogue, condition branching, player choices, or playing media.

Editor UI Basics

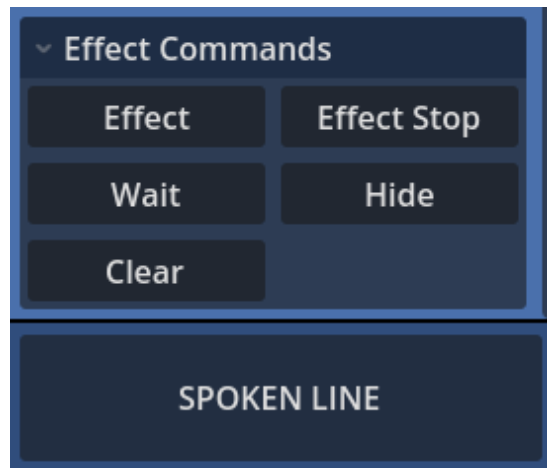


The large, central area of the editor UI displays all the lines inside a selected dialogue Block. An empty Comment line is automatically created in any new Block.

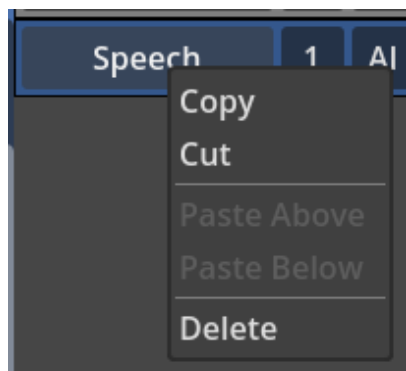
You can navigate Conversations and Blocks at the top of the screen:



Lines are added by clicking the buttons in the left area of the screen:



You can left-click the name of a line to select it, or you can right-click it to copy, cut, paste or delete a line:



Keyboard Shortcuts

Standard UI keyboard shortcuts are available:

CTRL+Click line name → Select multiple lines.

SHIFT+Click line name → Select a range of lines.

CTRL+C / CTRL+X → Copy or Cut a line (or multiple lines if selected).

CTRL+V / CTRL+SHIFT+V → Paste line(s) above or below. If multiple lines are selected when pasting, instances of the copied/cut lines will be pasted above or below each.

CTRL+Y / CTRL+SHIFT+Y → Undo / Redo adding, deleting or moving Conversations/Blocks/Lines.

CTRL+Click on a line insert button → Insert the new line at the top of the Block instead of at the end.

Also note that if you click a line insert button while lines are selected, this inserts a line directly before or after each of the selected lines instead of the end or top of the Block.

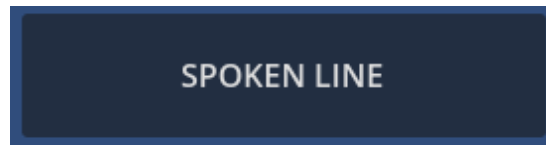
F4 / F5 → Quick Save / Quick Load

More shortcuts exist and are listed in the Intermediate documentation. These are just the basics you need to know at this point.

2. Example Dialogue

Spoken Lines

Start by clicking the large "Spoken Line" button in the bottom-left corner. This will add a new spoken line to the current Block.



Hide data fields

As you can see, the line is filled with buttons and text fields to input data. We won't be using all of them, so look at the very top of the screen and click the "Tags" and "Voices" buttons to hide those. This lightens things a bit.



Line Name and Index

Look at the two buttons on the far left of the line: "Speech" and "1". They respectively represent the name of the line, so you know what it does, and the index of the line inside the Block, for easy reference.



Speaker and Speech

Look at the spoken line again, and in the "Speaker" field, write "Alice" (without the quotation marks) or click the (:) button to the right and select "Alice" from the list.

In the "Speech" field, write "Hello, I'm Alice.".

That's it, you've just written your first dialogue line!



If you accidentally click the ↩ button, the Writer UI will open and fill the screen. Don't panic: just click the "Close" button in the top-right corner.

The Writer is a dedicated UI for editing spoken lines, and it's very helpful when you have a lot of Spoken Lines. However, to keep things simple, we won't use it right now.

Portraits

Let's add a portrait for this line: you could type a file name in the Portrait field, but an easier way is to simply select a file from the list.

Click the (:) button to the right of the Portrait field, and if you've setup everything correctly, a list with a few portraits should appear.

If nothing appears, you either placed the Candy_DE folder in the wrong location, you provided the wrong path to your Godot project in the Candy Dialogue Creator settings, you didn't write "Alice" correctly in the speaker field, or you deleted the portrait files.

If you do see a list, test what happens when you hover a portrait file with the mouse: it displays a preview!

Select the portrait file "Happy.png" by clicking on it, and it will automatically be added to the Portrait field. Hover the field, and you can see the preview again, in case you want to quickly check that it fits the mood of the line!

Don't worry about the ► button to the left of the Portrait field: it indicates whether you want animated portraits to play automatically, or to pause on the first frame. It's enabled by default, since we assume this is what users would most often choose. However, you don't need to disable it right now: since the default portraits are not animated, the ► button will be ignored.

Dispositions

Now let's make Alice say the line only if she likes the player: in the Disposition field, write "Like" (without the quotation marks) or click the (:) button and select "Like" from the list.



Don't worry about the trailing comma and space if you select from the list: it's automatically added in case you want to add multiple dispositions, but Candy DE will ignore it when processing the line.

Dispositions are an easy and quick way to make certain spoken lines conditional on the speaker's mood or disposition towards the player. Simply write a disposition (e.g. "Like") in the "Dispositions" field and, if the speaker's disposition matches in the game when the line is processed, the line will be displayed. If it's not a match, the line will be skipped.

If you hover the Disposition field long enough with the mouse, a tooltip will appear with a lot of additional information.

Don't worry about it for now, this is advanced stuff that you don't need to know about for basic dialogues. In fact, even advanced dialogues won't use all of these options. Like most Candy DE features, it's there if you need it, and it can be ignored entirely if you don't have a use for it.

Character limit

Finally, look at the far right of the spoken line, and notice a colored number. This tells you whether you've exceeded the maximum recommended character limit. This is helpful when your dialogue display UI supports a limited number of characters.



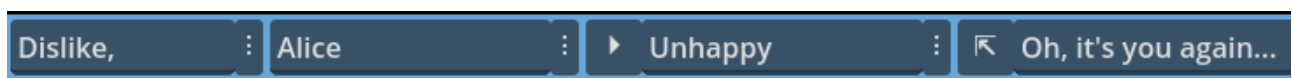
A green number indicates the characters remaining before you reach the limit, a yellow number means you're close to reaching the limit and you should consider splitting the text over another line, and a red number indicates by how many characters you've exceeded the limit.

The numbers are customizable. By default, the limit is set to 250 and the number turns yellow when you're 50 characters away from the limit.

Extra Line

To make sure you understood the basics of spoken lines, add a second line for Alice. Once again, choose her as the speaker, make her say "Oh, it's you again...", and choose the "Unhappy" portrait.

In the Disposition field, write or select "Dislike". This will demonstrate how dispositions work when we test the dialogue later: the first line (with disposition "Like") will show, but the second line (disposition "Dislike") won't show because Alice's default disposition is set to "Like" in Candy DE's database.



Command Lines

Let's now learn about command lines: look at the menu on the left of the screen with the command line buttons. Command lines might seem technical, but they're very easy to use once you understand what they do.

Control commands

Control commands are used for changing variables, calling functions, or emitting signals.



Click the "Name" button to add the §Name command to your dialogue.

In the code, commands are prefixed with the '\$' symbol so the engine can process them differently from Spoken Lines. The documentation (like this guide) follows this convention as well for clarity.

In the §Name command's Reference field, write "Alice" again (without quotation marks) or select Alice from the (:) list. In the Value field, write a new name for Alice (e.g. "Alice Turner" or "Alice the Evil Overlord").



The Name command will change how Alice's name is displayed when she speaks. However, it does not change her reference: you will still write "Alice" inside dialogue lines.

Notice how the §Name command line is green, while the Speech line is blue? These colors will make it easy to quickly identify lines by category when you browse through your dialogues for review or editing.

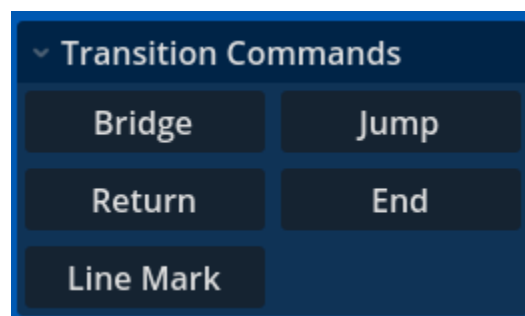
Comments

If you'd like to try, feel free to also add a Comment line by clicking the "Comment" button in the Control Commands menus on the left. Comments are only for helping you write and edit dialogues, they have no effect in your game. Write whatever you want in the Comment field, and change the text color by clicking the "Color" button. Just a small convenience feature.

Comment	3	Color	This is a blue test comment.
Comment	4	Color	This comment is yellow.
Comment	5	Color	So many colors!!!

Transition commands

Transition commands handle dialogue flow: they are used for switching to another Block during dialogue, or to End the dialogue.



There are two options for transitioning forward: \$Bridge and \$Jump.

\$Bridge is a momentary detour in the dialogue: Candy DE will switch to the indicated Block, process its lines, and then come back to the previous Block so it can continue where it left off.

\$Jump is a permanent change in the dialogue: Candy DE will not return to the previous Block after processing the new Block.

\$Return transitions backward: if a Block was reached through a \$Bridge transition, \$Return will return to that Block, and continue where it left off (i.e. after the \$Bridge command).

\$Line Mark acts as a line reference: \$Bridge and \$Jump can transition to a specific line using line indexes, but indexes are fragile (they'll change if you add or remove lines). Instead, it's much more reliable to target a \$Line Mark's reference.

\$End just ends the dialogue (more on that later).

Add a \$Jump command, as we don't want to return to Block_1, and edit its data like so:

Leave the Conversation field empty: Candy DE will automatically default to the same Conversation that the \$Jump command is located in.

In the Block field, select Block_2 (if you leave the Block field empty, Candy DE also defaults to the same Block the \$Jump command is located in).

Leave the Line field empty: it's only used when transitioning to a specific line inside a Block. If you leave it empty, it transitions to line 0 -- the first line of the Block -- which is what we want to do.



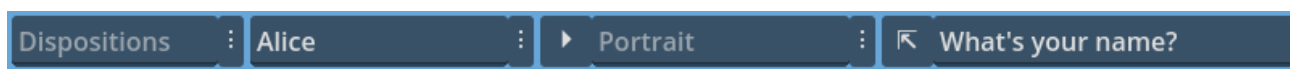
Finally, click the → button on the far right in the Jump line: this will automatically create the missing Block_2 inside Conversation_1 and make the UI switch to it. The lines on screen will disappear of course, since we're now in Block_2. If you ever want to create a Block without switching to it immediately, you can CTRL + Click the → button instead.

To navigate back to Block_1 at any time, you can use the Conversation and Block selector buttons at the very top of the screen.

More dialogue

Just to make sure everything worked when we test the dialogue, add a Spoken Line to Block_2.

Write "Alice" again as the speaker, leave the Portrait field empty to use her default portrait, and write "What's your name?" in the Speech field. Don't write a disposition either, this line shouldn't be conditional.



Since we changed Alice's display name with the §Name command earlier in Block_1, when the dialogue runs, her name on screen for this line will be "Alice Turner" (or whatever name you gave her).

Now, add a second Spoken Line to Block_2.

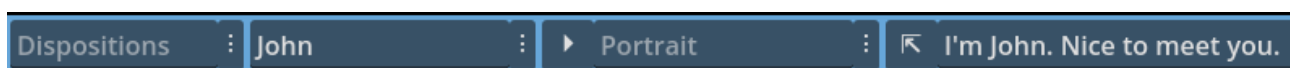
This time, write "John" in the Speaker field. John doesn't exist in Candy DE's 'actors' dictionary, so his name doesn't show up in the actor list if you click the (:) button.

When an actor doesn't exist in the dictionary, Candy DE will display the reference as the speaker's name for the line. It just won't have custom data for John the way it does for Alice, such as custom text color, dispositions, and so on. For minor characters who don't need an entry in your game's database, this is fine.

John also doesn't have portraits, because no folder exists for John in your project's resources. But you can write the name of a portrait file if you intend to create a folder with portraits for John later.

Even though 'John' isn't an entry in Candy DE's 'actors' dictionary, Candy DE can still look for a portrait folder for him: the two things are separate.

For John's Speech, write "I'm John. Nice to meet you."



Roles

Let's also learn about speaker roles while we're at it:

Add a \$Role command, and for the Role field, select "°civilian_1". In the Actor field, select "construction_worker".

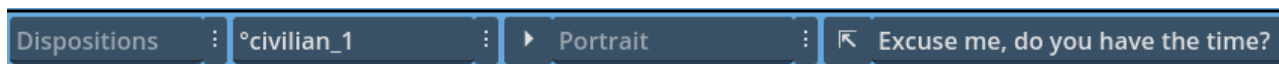


Once the \$Role command is setup, add a Spoken Line once more. In the Speaker field, write '°', then click the (:) button: notice that the list now displays role references instead of actor references.

Roles always start with the role symbol (default: '°'): this is how Candy DE knows whether it's dealing with a role reference or an actor reference.

For the Role command, you didn't need to write '°' in the Role field because the Candy Dialogue Creator knows it should only list roles there. But in Reference fields that can accept both actors and roles, typing '°' is required in order to display the list of roles.

Select "°civilian_1", and in the Speech field, write "Excuse me, do you have the time?"



Although evil_soldier has portraits and will be assigned to the °enemy_1 role by the earlier \$Role command, the portraits won't display in the portrait list since the assignment will occur at runtime.

You could manually write a portrait name in the Portrait field. Ideally, you would ensure any actors that can potentially be assigned to a role have portraits for the matching name.

If the written portrait doesn't exist for any actor assigned to a role, Candy DE will default to the **Default.png** portrait.

End

Since this is just a brief tutorial, we'll make the dialogue end here.

Add the \$End command, in the Transition Commands category:



Notice that it doesn't accept any data. That's because it serves a single purpose: ending dialogue. As soon as the dialogue engine runs this command, dialogue ends. Simple.

The §End command is a big red line, with a pattern of black diagonal stripes: this helps it stand out and visually mark where dialogue will stop.

Note that you don't always need to add the §End command. When the dialogue engine reaches the end of a Block, and if that Block was **not** reached through a §Bridge command, then the dialogue will end automatically because there's no lines left to process.

In this case, we reached Block_2 through a §Jump command, so the dialogue would end at the end of Block_2 anyway. That said, it's good practice to always add an §End command: it reduces the chances that you might forget to add it when needed, and it visually shows where dialogue is expected to end while you're editing.

3. Testing Dialogues

Exporting

Now that we have a minimal dialogue, click the "Test" button in the top-right corner of the UI. And just like that, your dialogue has been exported to your project resources!

TEST

To make sure it worked, go to your project folder, and from there, navigate to Candy_DE/Dialogues. You should see a TXT filed named "Test_Dialogue".

Note that this method is only for quickly exporting a test file. Any previous versions of the **test_dialogue.txt** file will be overwritten each time you click "Test". When your dialogue is complete and you're ready to make a proper export, the Export Menu, inside the Main Menu, offers additional options, including giving custom names to exported files.

Test Scene

Running dialogues

Now let's test your dialogue. Go back to the Godot Editor and open the scene named **Dialogue Test Scene.tscn** found in **Candy_DE/Scenes**.

Run the scene, and if you've followed all instructions correctly, your dialogue should immediately begin. That's it: two-click dialogue testing!

By default, the testing scene is configured to start dialogues at Conversation_1, Block_1.

To start at a different Conversation or Block, select the test scene's root node then, in the Inspector, change the **Conversation**, **Block** and **Line** exported variables to the values you want:

⚙ Testing_Scene.gd	
Dialogue	test_dialogue.txt
Conversation	Conversation_1
Block	Block_1
Line	0

Note that the test scene is empty and only starts dialogue. If you run it directly, you won't see game elements on screen, and some commands may have no effect (or may even crash). It can only display UI elements from the Candy_UI.

See the [Test Scene](#) guide for more information on how to fully test dialogues.

Success

If you did everything correctly, you'll see the following dialogue, in sequence:

Alice: Hello, I'm Alice.

Alice Turner: What's your name?

John: I'm John. Nice to meet you.

Construction Worker: Excuse me, do you have the time?

Errors

If the dialogue doesn't run:

1. Check that the starting Conversation and Block in the test scene match those in your dialogue.
2. Check that the **test_dialogue.txt** file is located in the **Candy_DE/Dialogues folder**.
3. Check that the **Candy_DE** folder is inside your Godot project's root folder.
4. In Candy Dialogue Creator, go to Main Menu → Settings → Project Files and verify that the Project Folder path is correct (i.e. it must point to your project's root folder).
5. Check that the "Custom Folders" box (to the right of the Project Folder path) is NOT ticked (the paths underneath must be grayed out).

Ending Dialogues

As previously mentioned, dialogues will end on their own when the last line has been processed or when encountering an §End command.

In both cases, the Godot Editor output area will print "Dialogue Ended" to let you know the dialogue ended properly and didn't stop due to an error.

4. Conclusion

You've now setup both Candy DE and Candy DC, and learned the basics of creating dialogues:

- Conversations and Blocks;
- Adding Lines;
- Speaker References (roles and actors);
- Portraits;
- The concept of Dispositions;
- Transitions;
- Control commands such as "Name" and "Role";
- Exporting dialogues for testing;
- Running a test dialogue in the Godot Editor;

You'll probably want to learn about a few of the following intermediate features:

- Alternate dialogue or portrait display modes (visual novels, speech bubbles, etc.);
- Adding actors, roles, or other information to Candy DE's database;
- More info on Dispositions;
- Conditions and branching;
- Player choices;
- Playing/displaying media;
- Calling functions or emitting signals;
- Displaying variables in spoken text;
- Customizing the UI;
- Using the Writer UI in Candy DC for even more effective writing;
- Resource folders (including custom paths);

Have a look in the subfolders in [Candy_DE/Documents/Guides](#) to find what you need!